CHARLESTON SOUTHERN UNIVERSITY

Civics Quiz

Senior Project Defense

Carla Contreras
B.S Computer Science
Charleston Southern University
Fall 2018

Advisor
Valerie Sessions, Ph.D.

TABLE OF CONTENTS

1. INTRODUCTION

The Senior Defense document documents and tracks all of the information required to effectively present and submit my Senior Project. This document was created during the 3rd phase of the Senior Project courses, CSCI 499. It is divided into several sections; Introduction, Background, Requirements, Code, Test Plan, and Details. Each section goes into more detail about specific things relating to the project.

1.1. Statement of Purpose

My motivation in completing this Senior Project is to demonstrate to myself and the Computer Science professors at Charleston Southern University (CSU) that I am able to apply concepts and ideas that I learned throughout my semesters at CSU to produce a project worthy from a senior preparing to graduate. I would like my project to be useful, easy to use, and have a purpose that could potentially in the future go out for people to use. Another one of my purpose is to provide documentation that will help anyone reading this document understand my project better and provide greater details on the design, implementation, process, and overall construction of my project.

2. BACKGROUND

When having to decide on a Senior Project idea, I wanted to focus on something that would mean something to me. During this time the issue of immigration started to come up more and more in the media. This gave me an idea that I could make an app that could help people learn something that would benefit them in the long run if it was successfully implemented. In the United States of America there are two ways to become a U.S Citizen, that is by birth or by naturalization. If you are born in this country, then you don't have to spend time here or work towards your citizenship because it is a birthright. However, if you were not born in this country you have to meet certain requirements to be able to apply for the citizenship test. These requirements are listed on the U.S Citizenship and Immigration Services (USCIS) website. Once you have applied and are granted a screening you must be able to pass a Citizenship Test. This test consists of 10 randomly selected questions from 100 questions. The applicants must be able to answer 6 out of the 10 questions to be able to pass. For my senior project I decided to make a Civics Quiz app for iPhones that allowed the user to practice the civic questions in two different forms. I made one for True or False and I created another one that is Multiple Choice. The user has a timer counting down how long they have to answer each question. If they get it wrong, they automatically get taken back to the main menu because the quiz is discontinued. If they get the questions right they are able to proceed to the next question. My Civics Quiz app is able to keep count of how many questions the individual has missed. If they miss one the background color to the screen changes. This is allowed to happen 5 times. If they miss more than 5 questions the quiz will stop again, and they will be returned to the menu. The

Civic Quiz app allows the user to record their high and more recent score. I added this so that the user could be motivated and not give up once they are returned back to the menu.

2.1. Research

With my Senior Project it was very important for me to do research and become familiar with the Citizenship procedures and the right tools to be able to implement my project. The very first step that I took in order to get started was to research what kind of test questions were given and know how they were going to be asked. Once I had this information I spent a few days playing around with online quizzes with those questions and becoming familiar with the questions themselves. Once this was already established I needed to decide what tools would help me accomplish my task the easiest. I ended up consulting an online iPhone programming book that suggested I used Swift, a general-purpose programming language developed by Apple Inc. for iOS, MacOS, tvOS, etc. Learning how to program with Swift was not extremely difficult but it did require a lot of self-discipline. I downloaded Xcode which is an IDE for MacOS. XCode has a Playgrounds which is an app that helps you learn to code and explore the things that you can do with Swift. Learning about view controllers in Swift is what took me the most time to figure out. The reason for this is because view controllers are what take care of what is being seen in on the phone screen and there are more than one so learning how to code the layout and the interactions for the view controllers was the most time-consuming experience that I had with my project. Overall the research was straight forward, and I really enjoyed using a programming language and software that have a lot of documentation on issues and processes.

3. REQUIREMENTS

The requirements listed are what I, myself used in order to complete this project. Since I have Apple products I used what would be easier to support my task. However, someone replicating this project could be able to use something different.

3.1. Hardware

- MacBook Pro (13-inch, 2017, Two Thunderbolt 3 ports)

3.2. Software

- XCode (Version 9.4.1)

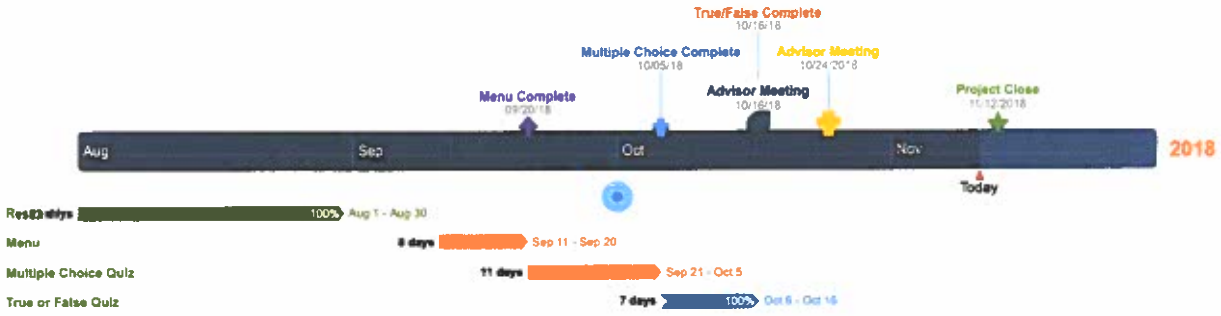
3.3. Programming Language

- Swift 4.1 (Released 9.17.18)

4. CODE

The following screenshots are categorized under each of their file names. I have included my Swift class files, Cocoa Touch class files, and my plist files.

4.1. Timeline



4.2. BotonRedondo.swift

```
//
//  BotonRedondo.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 9/30/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class BotonRedondo: UIButton {


    override func draw(_ rect: CGRect) {
        super.draw(rect)
        layer.cornerRadius = 5.0
        layer.masksToBounds = true

    }


}
```

4.3. EtiquetaRedonda.swift

```
//
//  EtiquetaRedonda.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 9/30/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class EtiquetaRedonda: UILabel {

    override func draw(_ rect: CGRect) {
        super.draw(rect)
        layer.cornerRadius = 5.0
        layer.masksToBounds = true
    }

    override func drawText(in rect: CGRect) {
        let nRectangle = rect.insetBy(dx: 8.0, dy: 8.0)
        super.drawText(in: nRectangle)
    }


}
```

4.4. Constantes+Extenciones.swift

```
//
//  Constantes+Extenciones.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 9/30/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import Foundation
import UIKit

let opcionMultiplePuntacionAltaID = "OpcionMultiplePuntacionAltaID"
let opcionMultiplePuntacionRecienteID = "opcionMultiplePuntacionRecienteID"

let trueFalsePuntacionAltaID = "TrueFalsePuntacionAltaID"
let trueFalsePuntacionRecienteID = "TrueFalsePuntacionRecienteID"

//let verdeP = UIColor(red: 46/255, green: 204/255, blue: 113/255, alpha: 1.0)
//let naranjaP = UIColor(red: 230/255, green: 126/255, blue: 34/255, alpha:
 1.0)
//let rojoP = UIColor(red: 231/255, green: 76/255, blue: 60/255, alpha: 1.0)


let verdeThanksgiving = UIColor(red: 127/255, green: 176/255, blue: 105/255,
 alpha: 1.0)
let naranjaThanksgiving = UIColor(red: 211/255, green: 97/255, blue: 53/255,
 alpha: 1.0)
let rojoThanksgiving = UIColor(red: 111/255, green: 29/255, blue: 27/255,
 alpha: 1.0)

let verdeFR = UIColor(red: 68/255, green: 204/255, blue: 34/255, alpha: 1.0)
let naranFR = UIColor(red: 241/255, green: 80/255, blue: 37/255, alpha: 1.0)
let rojoFR = UIColor(red: 227/255, green: 23/255, blue: 10/255, alpha: 1.0)
```

4.5. CargadorDeQuiz.swift

```
//
//  CargadorDeQuiz.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 10/3/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import Foundation

struct PreguntaOpcionMultiple {
    let pregunta: String
    let respuestaCorecta: String
    let respuestas: [String]
}

struct TF {
    let pregunta: String
    let respuestaCorecta: String
}


enum FalloCargador: Error {
    case falloDict, falloCamino
}


class CargadorDeQuiz {



    public func CargarTF(forQuiz quizName: String) throws -> [TF] {
        var preguntas = [TF]()
        if let camino = Bundle.main.path(forResource: quizName, ofType:
         "plist") {
            if let dict = NSDictionary(contentsOfFile: camino) {
                let tempArray: Array = dict["Questions"]! as!
                 [Dictionary<String,AnyObject>]
                for dictionary in tempArray {
                    let questionToAdd = TF(pregunta: dictionary["Question"] as!
                     String, respuestaCorecta: dictionary["CorrectAnswer"] as!
                     String)
                    preguntas.append(questionToAdd)
                }
                preguntas.shuffle();
                return preguntas
            } else {
                throw FalloCargador.falloDict
            }
        } else {
            throw FalloCargador.falloCamino
```

```
        }
    }


    //NO LE HAGAS NADA A ESTO POR ACCIDENTE
    public func cargarOpcionMultipleQuiz(paraQuiz quizNombre: String) throws ->
     [PreguntaOpcionMultiple] {
        var preguntas = [PreguntaOpcionMultiple]()
        if let camino = Bundle.main.path(forResource: quizNombre, ofType:
         "plist") {
            if let dict = NSDictionary(contentsOfFile: camino) {
                let tempArray: Array = dict["Questions"]! as!
                 [Dictionary<String,AnyObject>]
                for dictionary in tempArray {
                    let questionToAdd = PreguntaOpcionMultiple(pregunta:
                     dictionary["Question"] as! String, respuestaCorecta:
                     dictionary["CorrectAnswer"] as! String, respuestas:
                     dictionary["Answers"] as! [String])
                    preguntas.append(questionToAdd)
                }
                preguntas.shuffle()
                return preguntas
            } else {
                throw FalloCargador.falloDict
            }
        } else {
            throw FalloCargador.falloCamino
        }
    }


}

// https://www.reddit.com/r/swift/comments/7ix8z7/shuffle_an_array/
// https://en.wikipedia.org/wiki/Fisher-Yates_shuffle
extension Array {
    mutating func shuffle() {
        for i in indices.reversed() {
            let j = arc4random_uniform( numericCast(i+1) )
            swapAt(i, numericCast(j))
        }
    }

    func shuffled() -> Array {
        var newArray = self
        newArray.shuffle()
        return newArray
    }
}
```

4.6. ParteAlertaQuiz.swift

```
//
//  ParteAlertaQuiz.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 10/3/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class ParteAlertaQuiz: UIView {

    private let parteAlerta = UIView()
    private let etiquetaTitulo = EtiquetaRedonda()
    private let etiquetaMensaje = EtiquetaRedonda()
    let cerrarBoton = UIButton()

    init(conTitulo titulo: String, yMensaje mensaje: String, colors: [UIColor])
     {
        super.init(frame: CGRect.zero)
        etiquetaTitulo.text = titulo
        etiquetaMensaje.text = mensaje
        parteAlerta.backgroundColor = colors[0]
        cerrarBoton.backgroundColor = colors[1]
        vistaDeDiseno()
    }

    func vistaDeDiseno() {
        self.backgroundColor = UIColor(red: 0.2, green: 0.2, blue: 0.2, alpha:
         0.85)
        self.translatesAutoresizingMaskIntoConstraints = false

        parteAlerta.translatesAutoresizingMaskIntoConstraints = false
        self.addSubview(parteAlerta)

        etiquetaTitulo.translatesAutoresizingMaskIntoConstraints = false
        etiquetaMensaje.translatesAutoresizingMaskIntoConstraints = false
        parteAlerta.addSubview(etiquetaTitulo)
        parteAlerta.addSubview(etiquetaMensaje)
        etiquetaTitulo.font = UIFont.boldSystemFont(ofSize: 30)
        etiquetaTitulo.adjustsFontSizeToFitWidth = true
        etiquetaTitulo.textColor = UIColor.white
        etiquetaTitulo.textAlignment = .center
        etiquetaMensaje.font = UIFont.boldSystemFont(ofSize: 20)
        etiquetaMensaje.numberOfLines = 2
        etiquetaMensaje.adjustsFontSizeToFitWidth = true
        etiquetaMensaje.textColor = UIColor.white
        etiquetaMensaje.textAlignment = .center

        cerrarBoton.translatesAutoresizingMaskIntoConstraints = false
        parteAlerta.addSubview(cerrarBoton)
```

```
        cerrarBoton.titleLabel?.font = UIFont.boldSystemFont(ofSize: 30)
        cerrarBoton.titleLabel?.adjustsFontSizeToFitWidth = true
        cerrarBoton.titleLabel?.textColor = UIColor.white
        cerrarBoton.setTitle("Continue", for: .normal)

        let reglas = [
            parteAlerta.heightAnchor.constraint(equalTo: self.heightAnchor,
             multiplier: 0.3),
            parteAlerta.widthAnchor.constraint(equalTo: self.widthAnchor,
             multiplier: 0.8),
            parteAlerta.centerXAnchor.constraint(equalTo: self.centerXAnchor),
            parteAlerta.centerYAnchor.constraint(equalTo: self.centerYAnchor),
            etiquetaTitulo.topAnchor.constraint(equalTo: parteAlerta.topAnchor,
             constant: 8.0),
            etiquetaTitulo.leadingAnchor.constraint(equalTo:
             parteAlerta.leadingAnchor),
            etiquetaTitulo.trailingAnchor.constraint(equalTo:
             parteAlerta.trailingAnchor),
            etiquetaTitulo.heightAnchor.constraint(equalTo:
             parteAlerta.heightAnchor, multiplier: 0.2),
            etiquetaMensaje.topAnchor.constraint(equalTo:
             etiquetaTitulo.bottomAnchor),
            etiquetaMensaje.leadingAnchor.constraint(equalTo:
             parteAlerta.leadingAnchor),
            etiquetaMensaje.trailingAnchor.constraint(equalTo:
             parteAlerta.trailingAnchor),
            etiquetaMensaje.bottomAnchor.constraint(equalTo:
             cerrarBoton.topAnchor),
            cerrarBoton.heightAnchor.constraint(equalTo:
             parteAlerta.heightAnchor, multiplier: 0.2),
            cerrarBoton.leadingAnchor.constraint(equalTo:
             parteAlerta.leadingAnchor),
            cerrarBoton.trailingAnchor.constraint(equalTo:
             parteAlerta.trailingAnchor),
            cerrarBoton.bottomAnchor.constraint(equalTo:
             parteAlerta.bottomAnchor)
        ]

        NSLayoutConstraint.activate(reglas)
    }

    required init?(coder aDecoder: NSCoder) {
        fatalError("init(coder:) has not been implemented")
    }

}
```

4.7. MenuViewController.swift

```
//
//  ViewController.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 9/30/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class MenuViewController: UIViewController {

    private let parteContenido = UIView()
    private let parteLogo = UIImageView()
    private let parteBoton = UIView()
    private var botonesDeQuiz = [BotonRedondo]()
    private let partePuntacion = UIView()
    private let etiquetaTitulo = UILabel()
    private let etiquetaPuntacionReciente = UILabel()


    private let etiquetaPuntacionAlta = UILabel()

    private let titulos = [
        "Multiple Choice",
        "True or False"
    ]

    private var puntacionesRecientes = [Int]()
    private var puntacionesAltas = [Int]()
    private var puntacionIndex = 0
    private var reloj = Timer()

    private var midXReglas: [NSLayoutConstraint]!
    private var izqReglas: [NSLayoutConstraint]!
    private var derReglas: [NSLayoutConstraint]!

    override func viewDidLoad() {
        super.viewDidLoad()
        navigationController?.navigationBar.barStyle = .blackTranslucent
        navigationController?.navigationBar.tintColor = UIColor.white
       // navigationController?.navigationBar.tintColor = UIColor(red: 88/255,
        green: 106/255, blue: 106/255, alpha: 1.0)
         view.backgroundColor = UIColor(displayP3Red: 7/255, green: 7/255,
          blue: 7/255, alpha: 1.0)


       // view.backgroundColor = UIColor(hue: 50/255, saturation: 9/255,
        brightness: 51/255, alpha: 1.0)
        vistaDeDiseno()
    }
```

```
override func viewWillAppear(_ animated: Bool) {
    navigationController?.navigationBar.isHidden = true
    actualizarPuntaciones()
}

func actualizarPuntaciones() {
    puntacionesRecientes = [
        UserDefaults.standard.integer(forKey:
         opcionMultiplePuntacionRecienteID),
        UserDefaults.standard.integer(forKey: trueFalsePuntacionRecienteID)
    ]

    puntacionesAltas = [
        UserDefaults.standard.integer(forKey:
         opcionMultiplePuntacionAltaID),
        UserDefaults.standard.integer(forKey: trueFalsePuntacionAltaID)
    ]
}

func vistaDeDiseno() {
    parteContenido.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(parteContenido)

    parteLogo.translatesAutoresizingMaskIntoConstraints = false
    parteContenido.addSubview(parteLogo)
    parteLogo.image = UIImage(named: "logo")
```

```
parteBoton.translatesAutoresizingMaskIntoConstraints = false
parteContenido.addSubview(parteBoton)

for (index, titulo) in titulos.enumerated() {
    let boton = BotonRedondo()
    boton.translatesAutoresizingMaskIntoConstraints = false
    parteBoton.addSubview(boton)
  //  boton.backgroundColor = UIColor(red: 52/255, green: 152/255,
   blue: 219/255, alpha: 1.0)
    boton.backgroundColor = UIColor(red: 31/255, green: 36/255, blue:
     33/255, alpha: 1.0)

   // boton.backgroundColor = UIColor(hue: 255/255, saturation: 4/255,
    brightness: 73/255, alpha: 1.0)
    boton.titleLabel?.font = UIFont.boldSystemFont(ofSize: 20)
    boton.setTitle(titulo, for: .normal)
    boton.tag = index
    boton.addTarget(self, action: #selector(botonManager),
     for: .touchUpInside)
    botonesDeQuiz.append(boton)
}

partePuntacion.translatesAutoresizingMaskIntoConstraints = false




parteContenido.addSubview(partePuntacion)

etiquetaTitulo.translatesAutoresizingMaskIntoConstraints = false
etiquetaPuntacionReciente.translatesAutoresizingMaskIntoConstraints =
 false
etiquetaPuntacionAlta.translatesAutoresizingMaskIntoConstraints = false
partePuntacion.addSubview(etiquetaTitulo)
partePuntacion.addSubview(etiquetaPuntacionReciente)
partePuntacion.addSubview(etiquetaPuntacionAlta)

etiquetaTitulo.textAlignment = .center
etiquetaTitulo.font = UIFont.boldSystemFont(ofSize: 30)
etiquetaTitulo.textColor = UIColor.white
// etiquetaTitulo.textColor = UIColor(red: 88/255, green: 106/255, blue:
106/255, alpha: 1.0)
etiquetaPuntacionReciente.font = UIFont.boldSystemFont(ofSize: 20)
etiquetaPuntacionReciente.textColor = UIColor.white
etiquetaPuntacionAlta.font = UIFont.boldSystemFont(ofSize: 20)
etiquetaPuntacionAlta.textColor = UIColor.white

etiquetaTitulo.text = titulos[puntacionIndex]
etiquetaPuntacionReciente.text = "Most Recent: " +
 String(UserDefaults.standard.integer(forKey:
 opcionMultiplePuntacionRecienteID))
etiquetaPuntacionAlta.text = "Highscore: " +
 String(UserDefaults.standard.integer(forKey:
 opcionMultiplePuntacionAltaID))
```

```
let reglas = [

    //makes sure that it's using the entire content view.
    parteContenido.topAnchor.constraint(equalTo:
     view.safeAreaLayoutGuide.topAnchor, constant: 8.0),
    parteContenido.leadingAnchor.constraint(equalTo:
     view.leadingAnchor),
    parteContenido.trailingAnchor.constraint(equalTo:
     view.trailingAnchor),
    parteContenido.bottomAnchor.constraint(equalTo: view.bottomAnchor),

    //
    parteLogo.topAnchor.constraint(equalTo: parteContenido.topAnchor,
     constant: 20.0),
    parteLogo.widthAnchor.constraint(equalTo:
     parteContenido.widthAnchor, multiplier: 0.6),
    parteLogo.centerXAnchor.constraint(equalTo:
     parteContenido.centerXAnchor),
    parteLogo.heightAnchor.constraint(equalTo:
     parteContenido.heightAnchor, multiplier: 0.2),

    parteBoton.topAnchor.constraint(equalTo: parteLogo.bottomAnchor,
     constant: 20.0),




    parteBoton.bottomAnchor.constraint(equalTo:
     partePuntacion.topAnchor, constant: -20.0),
    parteBoton.widthAnchor.constraint(equalTo:
     parteContenido.widthAnchor, multiplier: 0.6),
    parteBoton.centerXAnchor.constraint(equalTo:
     parteContenido.centerXAnchor),

    botonesDeQuiz[0].topAnchor.constraint(equalTo:
     parteBoton.topAnchor, constant: 8.0),
    botonesDeQuiz[0].bottomAnchor.constraint(equalTo:
     botonesDeQuiz[1].topAnchor, constant: -8.0),
    botonesDeQuiz[0].leadingAnchor.constraint(equalTo:
     parteBoton.leadingAnchor),
    botonesDeQuiz[0].trailingAnchor.constraint(equalTo:
     parteBoton.trailingAnchor),
    botonesDeQuiz[1].bottomAnchor.constraint(equalTo:
     parteBoton.bottomAnchor, constant: -8.0),
    botonesDeQuiz[1].leadingAnchor.constraint(equalTo:
     parteBoton.leadingAnchor),
    botonesDeQuiz[1].trailingAnchor.constraint(equalTo:
     parteBoton.trailingAnchor),
    botonesDeQuiz[0].heightAnchor.constraint(equalTo:
     botonesDeQuiz[1].heightAnchor),

    partePuntacion.bottomAnchor.constraint(equalTo:
     parteContenido.bottomAnchor, constant: -40.0),
    partePuntacion.widthAnchor.constraint(equalTo:
```

```
            parteContenido.heightAnchor, multiplier: 0.3),
//          partePuntacion.centerXAnchor.constraint(equalTo:
 parteContenido.centerXAnchor),

            etiquetaTitulo.topAnchor.constraint(equalTo:
             partePuntacion.topAnchor, constant: 8.0),
            etiquetaTitulo.leadingAnchor.constraint(equalTo:
             partePuntacion.leadingAnchor),
            etiquetaTitulo.trailingAnchor.constraint(equalTo:
             partePuntacion.trailingAnchor),
            etiquetaTitulo.bottomAnchor.constraint(equalTo:
             etiquetaPuntacionReciente.topAnchor, constant: -8.0),

            etiquetaPuntacionReciente.leadingAnchor.constraint(equalTo:
             partePuntacion.leadingAnchor),
            etiquetaPuntacionReciente.trailingAnchor.constraint(equalTo:
             partePuntacion.trailingAnchor),
            etiquetaPuntacionReciente.bottomAnchor.constraint(equalTo:
             etiquetaPuntacionAlta.topAnchor, constant: -8.0),

            etiquetaPuntacionAlta.leadingAnchor.constraint(equalTo:
             partePuntacion.leadingAnchor),




            etiquetaPuntacionAlta.trailingAnchor.constraint(equalTo:
             partePuntacion.trailingAnchor),
            etiquetaPuntacionAlta.bottomAnchor.constraint(equalTo:
             partePuntacion.bottomAnchor, constant: -8.0),

            etiquetaTitulo.heightAnchor.constraint(equalTo:
             etiquetaPuntacionReciente.heightAnchor),

            etiquetaPuntacionReciente.heightAnchor.constraint(equalTo:
             etiquetaPuntacionAlta.heightAnchor)

        ]

        midXReglas = [partePuntacion.centerXAnchor.constraint(equalTo:
         parteContenido.centerXAnchor)]
        izqReglas = [partePuntacion.trailingAnchor.constraint(equalTo:
         parteContenido.leadingAnchor)]
        derReglas = [partePuntacion.leadingAnchor.constraint(equalTo:
         parteContenido.trailingAnchor)]

        NSLayoutConstraint.activate(reglas)
        NSLayoutConstraint.activate(midXReglas)

        reloj = Timer.scheduledTimer(timeInterval: 3.0, target: self, selector:
         #selector(proximasPuntaciones), userInfo: nil, repeats: true)

    }
```

```
    @objc func botonManager(sender: BotonRedondo) {
        var vc: UIViewController?
        switch sender.tag {
        case 0:
            vc = OpcionMultipleViewController()
        case 1:
            vc = TrueOrFalseViewController()
        default:
            break
        }
        if let newVC = vc {
            navigationController?.pushViewController(newVC, animated: true)
        }
    }

    @objc func proximasPuntaciones() {
        puntacionIndex = puntacionIndex < (puntacionesRecientes.count - 1) ?
         puntacionIndex + 1 : 0
        UIView.animate(withDuration: 1.0, animations: {
            NSLayoutConstraint.deactivate(self.midXReglas)
            NSLayoutConstraint.activate(self.izqReglas)
            self.view.layoutIfNeeded()
        }) { (completion: Bool) in




                self.etiquetaTitulo.text = self.titulos[self.puntacionIndex]
                self.etiquetaPuntacionReciente.text = "Most Recent: " +
                 String(self.puntacionesRecientes[self.puntacionIndex])
                self.etiquetaPuntacionAlta.text = "HighScore: " +
                 String(self.puntacionesAltas[self.puntacionIndex])
                NSLayoutConstraint.deactivate(self.izqReglas)
                NSLayoutConstraint.activate(self.derReglas)
                self.view.layoutIfNeeded()
                UIView.animate(withDuration: 1.0, animations: {
                    NSLayoutConstraint.deactivate(self.derReglas)
                    NSLayoutConstraint.activate(self.midXReglas)
                    self.view.layoutIfNeeded()
                })
            }
    }


}
```

4.8. OpcionMultipleViewController.swift

```
//
//  OpcionMultipleViewController.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 10/1/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class OpcionMultipleViewController: UIViewController {

    private var mal = 0
    private let parteContenido = UIView()
    private var parteContenidoReglas: [NSLayoutConstraint]!

    private let partePregunta = UIView()
    private var partePreguntaReglas: [NSLayoutConstraint]!

    private let parteRespuesta = UIView()
    private var parteRespuestaReglas: [NSLayoutConstraint]!

    private let parteConteo = UIView()
    private var parteConteoReglas: [NSLayoutConstraint]!

    private let etiquetaPregunta = EtiquetaRedonda()
    private var etiquetaPreguntaReglas: [NSLayoutConstraint]!
    private let botonPregunta = BotonRedondo()
    private var botonPreguntaReglas: [NSLayoutConstraint]!

    private var botonesRespuestas = [BotonRedondo]()
    private var botonesRespuestasReglas: [NSLayoutConstraint]!

    private let parteProgreso = UIProgressView()
    private var parteProgresoReglas: [NSLayoutConstraint]!

    private let colorDeFondo = UIColor(red: 47/255, green: 37/255, blue: 4/255,
     alpha: 1.0)
    private let colorDeLayer = UIColor(red: 224/255, green: 159/255, blue:
     62/255, alpha: 1.0)

   // private let colorDeLayerUno = UIColor(red: 46/255, green: 55/255, blue:
    49/255, alpha: 1.0)
    private let colorDeLayerUno = UIColor(red: 137/255, green: 137/255, blue:
     82/255, alpha: 1.0)
    private let colorDeLayerDos = UIColor(red: 178/255, green: 148/255, blue:
     91/255, alpha: 1.0)
    private let colorDeLayerTres = UIColor(red: 206/255, green: 108/255, blue:
     71/255, alpha: 1.0)
    private let colorDeLayerCuatro = UIColor(red: 94/255, green: 48/255, blue:
     35/255, alpha: 1.0)
```

```
private let colorDeLayerCinco = UIColor(red: 219/255, green: 190/255, blue:
 161/255, alpha: 1.0)


private let cargadorDeQuiz = CargadorDeQuiz()
private var preguntaArr = [PreguntaOpcionMultiple]()
private var preguntaIndex = 0
private var preguntaCoriente: PreguntaOpcionMultiple!

private var reloj = Timer()
private var puntacion = 0
private var puntacionAlta = UserDefaults.standard.integer(forKey:
 opcionMultiplePuntacionAltaID)

private var parteAlertaQuiz: ParteAlertaQuiz?


override func viewDidLoad() {
    super.viewDidLoad()
    view.backgroundColor = colorDeFondo
    vistaDeDiseno()
}

override func viewWillAppear(_ animated: Bool) {
    navigationController?.navigationBar.isHidden = false
}


func vistaDeDiseno() {

    parteContenido.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(parteContenido)

    partePregunta.translatesAutoresizingMaskIntoConstraints = false
    parteContenido.addSubview(partePregunta)
    etiquetaPregunta.translatesAutoresizingMaskIntoConstraints = false
    partePregunta.addSubview(etiquetaPregunta)
    etiquetaPregunta.backgroundColor = colorDeLayer
    etiquetaPregunta.textColor = UIColor.white
    etiquetaPregunta.font = UIFont.boldSystemFont(ofSize: 30)
    etiquetaPregunta.textAlignment = .center
    etiquetaPregunta.numberOfLines = 4
    etiquetaPregunta.adjustsFontSizeToFitWidth = true
    botonPregunta.translatesAutoresizingMaskIntoConstraints = false
    partePregunta.addSubview(botonPregunta)
    botonPregunta.addTarget(self, action: #selector(botonPreguntaManager),
     for: .touchUpInside)
    botonPregunta.isEnabled = false

    parteRespuesta.translatesAutoresizingMaskIntoConstraints = false
    parteContenido.addSubview(parteRespuesta)
    for _ in 0...3 {
```

```
    let boton = BotonRedondo()
    botonesRespuestas.append(boton)
    boton.translatesAutoresizingMaskIntoConstraints = false
    parteRespuesta.addSubview(boton)
    boton.addTarget(self, action: #selector(botonRespuestaManager),
     for: .touchUpInside)
}

parteConteo.translatesAutoresizingMaskIntoConstraints = false
parteContenido.addSubview(parteConteo)
parteProgreso.translatesAutoresizingMaskIntoConstraints = false
parteConteo.addSubview(parteProgreso)
parteProgreso.transform = parteProgreso.transform.scaledBy(x: 1, y: 10)

parteContenidoReglas = [
    parteContenido.topAnchor.constraint(equalTo:
     view.safeAreaLayoutGuide.topAnchor),
    parteContenido.leadingAnchor.constraint(equalTo:
     view.leadingAnchor),
    parteContenido.trailingAnchor.constraint(equalTo:
     view.trailingAnchor),
    parteContenido.bottomAnchor.constraint(equalTo: view.bottomAnchor)
]

partePreguntaReglas = [
    partePregunta.topAnchor.constraint(equalTo:
     parteContenido.topAnchor, constant: 20.0),
    partePregunta.leadingAnchor.constraint(equalTo:
     parteContenido.leadingAnchor, constant: 20.0),
    partePregunta.trailingAnchor.constraint(equalTo:
     parteContenido.trailingAnchor, constant: -20.0),
    partePregunta.heightAnchor.constraint(equalTo:
     parteContenido.heightAnchor, multiplier: 0.4)
]

etiquetaPreguntaReglas = [
    etiquetaPregunta.topAnchor.constraint(equalTo:
     partePregunta.topAnchor),
    etiquetaPregunta.leadingAnchor.constraint(equalTo:
     partePregunta.leadingAnchor),
    etiquetaPregunta.trailingAnchor.constraint(equalTo:
     partePregunta.trailingAnchor),
    etiquetaPregunta.bottomAnchor.constraint(equalTo:
     partePregunta.bottomAnchor)
]

botonPreguntaReglas = [
    botonPregunta.topAnchor.constraint(equalTo:
     partePregunta.topAnchor),
    botonPregunta.leadingAnchor.constraint(equalTo:
     partePregunta.leadingAnchor),
```

```
        botonPregunta.trailingAnchor.constraint(equalTo:
         partePregunta.trailingAnchor),
        botonPregunta.bottomAnchor.constraint(equalTo:
         partePregunta.bottomAnchor)
    ]

    parteRespuestaReglas = [
        parteRespuesta.topAnchor.constraint(equalTo:
         partePregunta.bottomAnchor, constant: 20.0),
        parteRespuesta.leadingAnchor.constraint(equalTo:
         parteContenido.leadingAnchor, constant: 20.0),
        parteRespuesta.trailingAnchor.constraint(equalTo:
         parteContenido.trailingAnchor, constant: -20.0),
        parteRespuesta.heightAnchor.constraint(equalTo:
         parteContenido.heightAnchor, multiplier: 0.4)
    ]

    botonesRespuestasReglas = [
        botonesRespuestas[0].leadingAnchor.constraint(equalTo:
         parteRespuesta.leadingAnchor),
        botonesRespuestas[0].trailingAnchor.constraint(equalTo:
         botonesRespuestas[1].leadingAnchor, constant: -8.0),
        botonesRespuestas[0].topAnchor.constraint(equalTo:
         parteRespuesta.topAnchor),
        botonesRespuestas[0].bottomAnchor.constraint(equalTo:
         botonesRespuestas[2].topAnchor, constant: -8.0),
        botonesRespuestas[1].trailingAnchor.constraint(equalTo:
         parteRespuesta.trailingAnchor),
        botonesRespuestas[1].topAnchor.constraint(equalTo:
         parteRespuesta.topAnchor),
        botonesRespuestas[1].bottomAnchor.constraint(equalTo:
         botonesRespuestas[3].topAnchor, constant: -8.0),
        botonesRespuestas[2].leadingAnchor.constraint(equalTo:
         parteRespuesta.leadingAnchor),
        botonesRespuestas[2].trailingAnchor.constraint(equalTo:
         botonesRespuestas[3].leadingAnchor, constant: -8.0),
        botonesRespuestas[2].bottomAnchor.constraint(equalTo:
         parteRespuesta.bottomAnchor),
        botonesRespuestas[3].trailingAnchor.constraint(equalTo:
         parteRespuesta.trailingAnchor),
        botonesRespuestas[3].bottomAnchor.constraint(equalTo:
         parteRespuesta.bottomAnchor)
    ]

    for index in 1..<botonesRespuestas.count {

        botonesRespuestasReglas.append(botonesRespuestas[index].heightAnch
        or.constraint(equalTo: botonesRespuestas[index-1].heightAnchor))

        botonesRespuestasReglas.append(botonesRespuestas[index].widthAncho
        r.constraint(equalTo: botonesRespuestas[index-1].widthAnchor))
```

```
        }

        parteConteoReglas = [
            parteConteo.topAnchor.constraint(equalTo:
             parteRespuesta.bottomAnchor, constant: 20.0),
            parteConteo.leadingAnchor.constraint(equalTo:
             parteContenido.leadingAnchor, constant: 20.0),
            parteConteo.trailingAnchor.constraint(equalTo:
             parteContenido.trailingAnchor, constant: -20.0),
            parteConteo.bottomAnchor.constraint(equalTo:
             parteContenido.bottomAnchor, constant: -20.0)
        ]

        parteProgresoReglas = [
            parteProgreso.leadingAnchor.constraint(equalTo:
             parteConteo.leadingAnchor),
            parteProgreso.trailingAnchor.constraint(equalTo:
             parteConteo.trailingAnchor),
            parteProgreso.centerYAnchor.constraint(equalTo:
             parteConteo.centerYAnchor)
        ]

        NSLayoutConstraint.activate(parteContenidoReglas)
        NSLayoutConstraint.activate(partePreguntaReglas)
        NSLayoutConstraint.activate(etiquetaPreguntaReglas)
        NSLayoutConstraint.activate(botonPreguntaReglas)
        NSLayoutConstraint.activate(parteRespuestaReglas)
        NSLayoutConstraint.activate(botonesRespuestasReglas)
        NSLayoutConstraint.activate(parteConteoReglas)
        NSLayoutConstraint.activate(parteProgresoReglas)

        cargarPreguntas()
    }

    func cargarPreguntas() {
        do {
            preguntaArr = try cargadorDeQuiz.cargarOpcionMultipleQuiz(paraQuiz:
             "MultipleChoice")
            cargarProximaPregunta()
        } catch {
            switch error {
            case FalloCargador.falloDict:
                print("dicc no cargo")
            case FalloCargador.falloCamino:
                print("camino no es valido")
            default:
                print("deconocido")
            }
        }
    }
```

```
func cargarProximaPregunta() {
    preguntaCoriente = preguntaArr[preguntaIndex]
    ponerTitulosParaBotones()
}

func ponerTitulosParaBotones() {
    for (index,button) in botonesRespuestas.enumerated() {
        button.titleLabel?.lineBreakMode = .byWordWrapping
        button.setTitle(preguntaCoriente.respuestas[index], for: .normal)
        button.isEnabled = true
        button.backgroundColor = colorDeFondo
    }
    etiquetaPregunta.text = preguntaCoriente.pregunta
    empiezaReloj()
}

func empiezaReloj() {
    parteProgreso.progressTintColor = verdeThanksgiving
    parteProgreso.trackTintColor = UIColor.clear
    parteProgreso.progress = 1.0
    reloj = Timer.scheduledTimer(timeInterval: 0.01, target: self,
     selector: #selector(actualizarParteProgreso), userInfo: nil, repeats:
     true)
}

@objc func actualizarParteProgreso() {
    parteProgreso.progress -= 0.01/30
    if parteProgreso.progress <= 0 {
       terminoTiempo()
    } else if parteProgreso.progress <= 0.2 {
        parteProgreso.progressTintColor = rojoThanksgiving
    } else if parteProgreso.progress <= 0.5 {
        parteProgreso.progressTintColor = naranjaThanksgiving
    }
}

func terminoTiempo() {
    reloj.invalidate()
    muestraAlerta(porRazon: 0)
    for boton in botonesRespuestas {
        boton.isEnabled = false
    }
}

@objc func botonPreguntaManager() {
    botonPregunta.isEnabled = false
    preguntaIndex += 1
    preguntaIndex < preguntaArr.count ? cargarProximaPregunta() :
     muestraAlerta(porRazon: 2)
}
```

```
@objc func botonRespuestaManager(_ sender: BotonRedondo) {
    reloj.invalidate()
    if sender.titleLabel?.text == preguntaCoriente.respuestaCorecta {
        puntacion += 1
        etiquetaPregunta.text = "Click to continue"
        botonPregunta.isEnabled = true
    } else {
        sender.backgroundColor = rojoThanksgiving
        mal = mal + 1

        if(mal == 1){
             etiquetaPregunta.backgroundColor = colorDeLayerUno
        }
        else if(mal == 2){
            etiquetaPregunta.backgroundColor = colorDeLayerDos
        }
        else if(mal == 3){
            etiquetaPregunta.backgroundColor = colorDeLayerTres
        }
        else if(mal == 4){
            etiquetaPregunta.backgroundColor = colorDeLayerCuatro
        }
        else if(mal == 5){
            etiquetaPregunta.backgroundColor = colorDeLayerCinco
        }


        if(mal > 5){
            muestraAlerta(porRazon: 1)
        }
        etiquetaPregunta.text = "Click to continue"
        botonPregunta.isEnabled = true
    }
    for boton in botonesRespuestas {
        boton.isEnabled = false
        if boton.titleLabel?.text == preguntaCoriente.respuestaCorecta {
            boton.backgroundColor = verdeThanksgiving
        }
    }
}

func muestraAlerta(porRazon razon: Int) {
    switch razon {
    case 0:
        parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You lost", yMensaje:
         "You ran out of time", colors: [colorDeFondo, colorDeLayer])
    case 1:
        parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You lost", yMensaje:
         "You picked the wrong answer", colors:
         [colorDeFondo,colorDeLayer])
    case 2:
```

```
            parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You won", yMensaje:
             "You have answered all questions", colors:
             [colorDeFondo,colorDeLayer])
        default:
            break
        }

        if let paq = parteAlertaQuiz {
        parteAlertaQuiz?.cerrarBoton.addTarget(self, action:
         #selector(cerrarAlerta), for: .touchUpInside)
        crearParteAlertaQuiz(conAlerta: paq)
        }

    }

    func crearParteAlertaQuiz(conAlerta alerta: ParteAlertaQuiz) {
        alerta.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(alerta)

        alerta.topAnchor.constraint(equalTo: view.topAnchor).isActive = true
        alerta.leadingAnchor.constraint(equalTo: view.leadingAnchor).isActive =
         true
        alerta.trailingAnchor.constraint(equalTo: view.trailingAnchor).isActive
         = true
        alerta.bottomAnchor.constraint(equalTo: view.bottomAnchor).isActive =
         true
    }


    @objc func cerrarAlerta() {
        if puntacion > puntacionAlta {
            puntacionAlta = puntacion
            UserDefaults.standard.set(puntacionAlta, forKey:
             opcionMultiplePuntacionAltaID)
        }
        UserDefaults.standard.set(puntacion, forKey:
         opcionMultiplePuntacionRecienteID)
        _ = navigationController?.popViewController(animated: true)
    }

    override func didMove(toParentViewController parent: UIViewController?) {
        super.didMove(toParentViewController: parent)
        if parent == nil {
            reloj.invalidate()
        }
    }
}
```

4.9. TrueOrFalseViewController.swift

```
//
//  OpcionMultipleViewController.swift
//  CarlaNayeliContreras
//
//  Created by Carla Contreras on 10/1/18.
//  Copyright © 2018 Carla Contreras. All rights reserved.
//

import UIKit

class TrueOrFalseViewController: UIViewController {

    private var mal = 0
    private let vistaContenido = UIView()
    private var vistaContenidoReglas: [NSLayoutConstraint]!

    private let vistaPregunta = UIView()
    private var vistaPreguntaReglas: [NSLayoutConstraint]!

    private let vistaRespuesta = UIView()
    private var vistaRespuestaReglas: [NSLayoutConstraint]!

    private let vistaConteo = UIView()
    private var vistaConteoReglas: [NSLayoutConstraint]!

    private let etiquetaPregunta = EtiquetaRedonda()
    private var etiquetaPreguntaRegla: [NSLayoutConstraint]!
    private let botonPregunta = BotonRedondo()
    private var botonPreguntaReglas: [NSLayoutConstraint]!

    private var botonesRespuesta = [BotonRedondo]()
    private var botonesRespuestaReglas: [NSLayoutConstraint]!

    private let parteProgreso = UIProgressView()
    private var parteProgresoReglas: [NSLayoutConstraint]!

    private let colorDeFondo = UIColor(red: 66/255, green: 17/255, blue:
     60/255, alpha: 1.0)
    private let colorDeLayer = UIColor(red: 97/255, green: 139/255, blue:
     37/255, alpha: 1.0)

    private let colorDeLayerUno = UIColor(red: 243/255, green: 167/255, blue:
     18/255, alpha: 1.0)
    private let colorDeLayerDos = UIColor(red: 64/255, green: 63/255, blue:
     76/255, alpha: 1.0)
    private let colorDeLayerTres = UIColor(red: 114/255, green: 24/255, blue:
     23/255, alpha: 1.0)
    private let colorDeLayerCuatro = UIColor(red: 53/255, green: 24/255, blue:
     8/255, alpha: 1.0)
    private let colorDeLayerCinco = UIColor(red: 71/255, green: 74/255, blue:
     72/255, alpha: 1.0)
```

```
private let cargadorDeQuiz = CargadorDeQuiz()

private var preguntaArr = [TF]()
private var preguntaIndex = 0
private var preguntaCoriente: TF!

private var reloj = Timer()
private var puntacion = 0
private var puntacionAlta = UserDefaults.standard.integer(forKey:
 trueFalsePuntacionRecienteID)

private var parteAlertaQuiz: ParteAlertaQuiz?

override func viewDidLoad() {
    super.viewDidLoad()
    view.backgroundColor = colorDeFondo
    vistaDeDiseno()
}

override func viewWillAppear(_ animated: Bool) {
    navigationController?.navigationBar.isHidden = false
}

func vistaDeDiseno() {
    vistaContenido.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(vistaContenido)


    vistaPregunta.translatesAutoresizingMaskIntoConstraints = false
    vistaContenido.addSubview(vistaPregunta)
    etiquetaPregunta.translatesAutoresizingMaskIntoConstraints = false
    vistaPregunta.addSubview(etiquetaPregunta)
    etiquetaPregunta.backgroundColor = colorDeLayer
    etiquetaPregunta.textColor = UIColor.black
    etiquetaPregunta.font = UIFont.boldSystemFont(ofSize: 30)
    etiquetaPregunta.textAlignment = .center
    etiquetaPregunta.numberOfLines = 4
    etiquetaPregunta.adjustsFontSizeToFitWidth = true
    botonPregunta.translatesAutoresizingMaskIntoConstraints = false
    vistaPregunta.addSubview(botonPregunta)
    botonPregunta.addTarget(self, action: #selector(preguntaBotonManager),
     for: .touchUpInside)
    botonPregunta.isEnabled = false

    vistaRespuesta.translatesAutoresizingMaskIntoConstraints = false
    vistaContenido.addSubview(vistaRespuesta)
    for index in 0...1 {
        let boton = BotonRedondo()
        botonesRespuesta.append(boton)
        boton.translatesAutoresizingMaskIntoConstraints = false
```

```
    vistaRespuesta.addSubview(boton)
    index == 0 ? boton.setTitle("TRUE", for: .normal) :
     boton.setTitle("FALSE", for: .normal)
    boton.addTarget(self, action: #selector(respuestaBotonManager),
     for: .touchUpInside)
}

vistaConteo.translatesAutoresizingMaskIntoConstraints = false
vistaContenido.addSubview(vistaConteo)
parteProgreso.translatesAutoresizingMaskIntoConstraints = false
vistaConteo.addSubview(parteProgreso)
parteProgreso.transform = parteProgreso.transform.scaledBy(x: 1, y: 10)

vistaContenidoReglas = [
    vistaContenido.topAnchor.constraint(equalTo:
     view.safeAreaLayoutGuide.topAnchor),
    vistaContenido.leadingAnchor.constraint(equalTo:
     view.leadingAnchor),
    vistaContenido.trailingAnchor.constraint(equalTo:
     view.trailingAnchor),
    vistaContenido.bottomAnchor.constraint(equalTo: view.bottomAnchor)
]

vistaPreguntaReglas = [
    vistaPregunta.topAnchor.constraint(equalTo:
     vistaContenido.topAnchor, constant: 20.0),
    vistaPregunta.leadingAnchor.constraint(equalTo:
     vistaContenido.leadingAnchor, constant: 20.0),
    vistaPregunta.trailingAnchor.constraint(equalTo:
     vistaContenido.trailingAnchor, constant: -20.0),
    vistaPregunta.heightAnchor.constraint(equalTo:
     vistaContenido.heightAnchor, multiplier: 0.4)
]

etiquetaPreguntaRegla = [
    etiquetaPregunta.topAnchor.constraint(equalTo:
     vistaPregunta.topAnchor),
    etiquetaPregunta.leadingAnchor.constraint(equalTo:
     vistaPregunta.leadingAnchor),
    etiquetaPregunta.trailingAnchor.constraint(equalTo:
     vistaPregunta.trailingAnchor),
    etiquetaPregunta.bottomAnchor.constraint(equalTo:
     vistaPregunta.bottomAnchor)
]

botonPreguntaReglas = [
    botonPregunta.topAnchor.constraint(equalTo:
     vistaPregunta.topAnchor),
    botonPregunta.leadingAnchor.constraint(equalTo:
     vistaPregunta.leadingAnchor),
```

```
        botonPregunta.trailingAnchor.constraint(equalTo:
         vistaPregunta.trailingAnchor),
        botonPregunta.bottomAnchor.constraint(equalTo:
         vistaPregunta.bottomAnchor)
    ]

    vistaRespuestaReglas = [
        vistaRespuesta.topAnchor.constraint(equalTo:
         vistaPregunta.bottomAnchor, constant: 20.0),
        vistaRespuesta.leadingAnchor.constraint(equalTo:
         vistaContenido.leadingAnchor, constant: 20.0),
        vistaRespuesta.trailingAnchor.constraint(equalTo:
         vistaContenido.trailingAnchor, constant: -20.0),
        vistaRespuesta.heightAnchor.constraint(equalTo:
         vistaContenido.heightAnchor, multiplier: 0.4)
    ]

    botonesRespuestaReglas = [
        botonesRespuesta[0].leadingAnchor.constraint(equalTo:
         vistaRespuesta.leadingAnchor),
        botonesRespuesta[0].trailingAnchor.constraint(equalTo:
         vistaRespuesta.trailingAnchor),
        botonesRespuesta[0].topAnchor.constraint(equalTo:
         vistaRespuesta.topAnchor),
        botonesRespuesta[0].bottomAnchor.constraint(equalTo:
         botonesRespuesta[1].topAnchor, constant: -8.0),
        botonesRespuesta[1].leadingAnchor.constraint(equalTo:
         vistaRespuesta.leadingAnchor),
        botonesRespuesta[1].trailingAnchor.constraint(equalTo:
         vistaRespuesta.trailingAnchor),
        botonesRespuesta[1].bottomAnchor.constraint(equalTo:
         vistaRespuesta.bottomAnchor),
        botonesRespuesta[0].heightAnchor.constraint(equalTo:
         botonesRespuesta[1].heightAnchor),
        botonesRespuesta[0].widthAnchor.constraint(equalTo:
         botonesRespuesta[1].widthAnchor)

    ]

    vistaConteoReglas = [
        vistaConteo.topAnchor.constraint(equalTo:
         vistaRespuesta.bottomAnchor, constant: 20.0),
        vistaConteo.leadingAnchor.constraint(equalTo:
         vistaContenido.leadingAnchor, constant: 20.0),
        vistaConteo.trailingAnchor.constraint(equalTo:
         vistaContenido.trailingAnchor, constant: -20.0),
        vistaConteo.bottomAnchor.constraint(equalTo:
         vistaContenido.bottomAnchor, constant: -20.0)
    ]

    parteProgresoReglas = [
```

```
            parteProgreso.leadingAnchor.constraint(equalTo:
             vistaConteo.leadingAnchor),
            parteProgreso.trailingAnchor.constraint(equalTo:
             vistaConteo.trailingAnchor),
            parteProgreso.centerYAnchor.constraint(equalTo:
             vistaConteo.centerYAnchor)
        ]

        NSLayoutConstraint.activate(vistaContenidoReglas)
        NSLayoutConstraint.activate(vistaPreguntaReglas)
        NSLayoutConstraint.activate(etiquetaPreguntaRegla)
        NSLayoutConstraint.activate(botonPreguntaReglas)
        NSLayoutConstraint.activate(vistaRespuestaReglas)
        NSLayoutConstraint.activate(botonesRespuestaReglas)
        NSLayoutConstraint.activate(vistaConteoReglas)
        NSLayoutConstraint.activate(parteProgresoReglas)

        cargarPreguntas()
    }

    func cargarPreguntas() {
        do {
            preguntaArr = try cargadorDeQuiz.CargarTF(forQuiz: "TrueOrFalse")
            cargarProximaPregunta()
        } catch {
            switch error {
            case FalloCargador.falloDict:
                print("dicc no cargo")
            case FalloCargador.falloCamino:
                print("camino no es valido")
            default:
                print("desconocido")
            }
        }
    }

    func cargarProximaPregunta() {
        preguntaCoriente = preguntaArr[preguntaIndex]
        ponerTitulosParaBotones()
    }

    func ponerTitulosParaBotones() {
        for boton in botonesRespuesta {
            boton.isEnabled = true
            boton.backgroundColor = colorDeLayer
            boton.setTitleColor(UIColor.black, for: .normal)
        }
        etiquetaPregunta.text = preguntaCoriente.pregunta
        empezarReloj()
    }
```

```
func empezarReloj() {
    parteProgreso.progressTintColor = verdeFR
    parteProgreso.trackTintColor = UIColor.clear
    parteProgreso.progress = 1.0
    reloj = Timer.scheduledTimer(timeInterval: 0.01, target: self,
     selector: #selector(actualizarVistaDeProgreso), userInfo: nil,
     repeats: true)
}

@objc func actualizarVistaDeProgreso() {
    parteProgreso.progress -= 0.01/30
    if parteProgreso.progress <= 0 {
        terminoTiempo()
    } else if parteProgreso.progress <= 0.2 {
        parteProgreso.progressTintColor = rojoFR
    } else if parteProgreso.progress <= 0.5 {
        parteProgreso.progressTintColor = naranFR
    }
}

func terminoTiempo() {
    reloj.invalidate()
    muestaAlerta(porRazon: 0)
}

@objc func preguntaBotonManager() {
    botonPregunta.isEnabled = false
    preguntaIndex += 1
    preguntaIndex < preguntaArr.count ? cargarProximaPregunta() :
     muestaAlerta(porRazon: 2)
}

@objc func respuestaBotonManager(_ sender: BotonRedondo) {
    reloj.invalidate()
    if sender.titleLabel?.text == preguntaCoriente.respuestaCorecta {
        puntacion += 1
        etiquetaPregunta.text = "Tap to continue"
        botonPregunta.isEnabled = true
    } else {
        sender.backgroundColor = rojoFR
        sender.setTitleColor(UIColor.black, for: .normal)
        mal = mal + 1
        if(mal == 1){
            etiquetaPregunta.backgroundColor = colorDeLayerUno
        }
        else if(mal == 2){
            etiquetaPregunta.backgroundColor = colorDeLayerDos
        }
        else if(mal == 3){
            etiquetaPregunta.backgroundColor = colorDeLayerTres
        }
```

```
        else if(mal == 4){
            etiquetaPregunta.backgroundColor = colorDeLayerCuatro
        }
        else if(mal == 5){
            etiquetaPregunta.backgroundColor = colorDeLayerCinco
        }
        if(mal > 5){
            muestaAlerta(porRazon: 1)
        }
        etiquetaPregunta.text = "Tap to continue"
        botonPregunta.isEnabled = true
    }
    for button in botonesRespuesta {
        button.isEnabled = false
        if button.titleLabel?.text == preguntaCoriente.respuestaCorecta {
            button.backgroundColor = verdeFR
            button.setTitleColor(UIColor.black, for: .normal)
        }
    }
}

func muestaAlerta(porRazon razon: Int) {
    switch razon {
    case 0:
        parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You lost", yMensaje:
         "You ran out of time", colors: [colorDeFondo,colorDeLayer])
    case 1:
        parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You lost", yMensaje:
         "You picked the wrong answer", colors:
         [colorDeFondo,colorDeLayer])
    case 2:
        parteAlertaQuiz = ParteAlertaQuiz(conTitulo: "You won", yMensaje:
         "You have answered all questions", colors:
         [colorDeFondo,colorDeLayer])
    default:
        break
    }

    if let pam = parteAlertaQuiz {
        parteAlertaQuiz?.cerrarBoton.addTarget(self, action:
         #selector(cerrarAlerta), for: .touchUpInside)
        parteAlertaQuiz?.cerrarBoton.setTitleColor(UIColor.darkGray,
         for: .normal)
        crearVistaAlertaQuiz(conAlerta: pam)
    }
}

func crearVistaAlertaQuiz(conAlerta alerta: ParteAlertaQuiz) {
    alerta.translatesAutoresizingMaskIntoConstraints = false
    view.addSubview(alerta)
```

```
            alerta.topAnchor.constraint(equalTo: view.topAnchor).isActive = true
            alerta.leadingAnchor.constraint(equalTo: view.leadingAnchor).isActive =
             true
            alerta.trailingAnchor.constraint(equalTo: view.trailingAnchor).isActive
             = true
            alerta.bottomAnchor.constraint(equalTo: view.bottomAnchor).isActive =
             true
        }

        @objc func cerrarAlerta() {
            if puntacion > puntacionAlta {
                puntacionAlta = puntacion
                UserDefaults.standard.set(puntacionAlta, forKey:
                 trueFalsePuntacionAltaID)
            }
            UserDefaults.standard.set(puntacion, forKey:
             trueFalsePuntacionRecienteID)
            _ = navigationController?.popViewController(animated: true)
        }

        override func didMove(toParentViewController parent: UIViewController?) {
            super.didMove(toParentViewController: parent)
            if parent == nil {
                reloj.invalidate()
            }
        }


    }
```

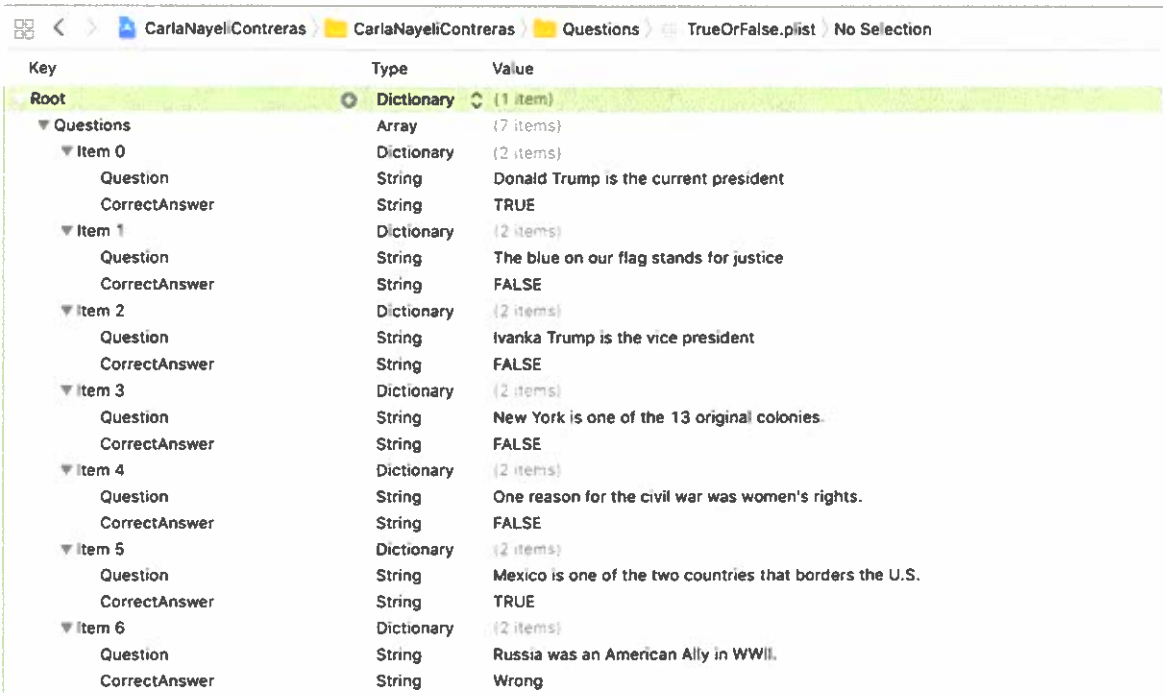
4.10. MultipleChoice.plist

The property list files can be inputted two different ways, one of those ways being with source code. The following screenshots are not the entire source code file but a sample of what the document looks like and how it is set up.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple Computer//DTD PLIST 1.0//EN" "http://
 www.apple.com/DTDs/PropertyList-1.0.dtd">
<!--
   OpcionMultiple.plist
   CarlaNayeliContreras

   Created by Carla Contreras on 10/3/18.
   Copyright (c) 2018 Carla Contreras. All rights reserved.
-->
<plist version="1.0">
    <dict>
        <key>Questions</key>
        <array>
            <dict>
                <key>Answers</key>
                <array>
                    <string>The U.S. Constitution</string>
                    <string>Uniform Code of Military Justice (UCMJ)</string>
                    <string>The Declaration of independence</string>
                    <string>The Supreme Court</string>
                </array>
                <key>CorrectAnswer</key>
                <string>The U.S. Constitution</string>
                <key>Question</key>
                <string>What is the supreme law of the land?</string>
            </dict>


            <dict>
                <key>Answers</key>
                <array>
                    <string>New Year's Day, Groundhog Day</string>
                    <string>Labor Day, Lady Gaga Day</string>
                    <string>Thanksgiving, Christmas</string>
                    <string>Memorial Day, Bring Your Daughter to Work Day</
                     string>
                </array>
                <key>CorrectAnswer</key>
                <string>Thanksgiving, Christmas</string>
                <key>Question</key>
                <string>Which are two national U.S. holidays?</string>
            </dict>



        </array>



    </dict>
</plist>
```

4.11. TrueOrFalse.plist

The other way that plist files can be inputted is manually. It is easy to create different sections and subsections of what you want your code to key to. The following screenshots are not the entire plist file but a sample of what the document looks like and how it is set up if you were to do it manually.



5. TEST PLAN

5.1. Introduction

The Test Plan documents and tracks the necessary information required to effectively define the approach to be used in the testing of my Civics Quiz Application. Its intended audience is my advisor, Valerie Sessions. There were two ways to test my application. I did self-testing whenever I completed each of the test. I checked functionality of buttons and features that I implemented into the app. Some of the test passed while other did not. The second test was having a classmate test my application. I sat down with him and ran the simulation of my quiz. He went through the entire application and gave me feedback on the same things that I tested as well as his opinion on things.

5.2. Pass/Fail Criteria

The pass/fail criteria for the self-testing is to check the functionality of each item 5 times and then see what the overall results were of all of the test. The results should be the same and I should be able to reproduce them all 5 times if they are failed or passed. For the test that my classmate did the criteria would be dependent on the information that he gives back to me and what feedback I receive from the same test that I ran to see if the results were indeed the same.

5.3. Test Results

Self-testing results:

	Test Date	Pass	Fail
Make sure Recent and High score update after quiz	OCTOBER 30, 2018	PASSED	
Check button functionality	NOVEMBER 02, 2018	PASSED	
Make sure prompts appear	OCTOBER 30, 2018	PASSED	
Run through True or False Quiz successfully	OCTOBER 30, 2018	PASSED	
Run Through Multiple Choice Quiz successfully	OCTOBER 30, 2018		FAIL, THERE IS AN ERROR THAT OCCURS RANDOMLY
Checked timer did not get stuck	OCTOBER 30, 2018	PASSED	

Classmate results:

	Test Date	Pass	Fail
Make sure Recent and High score update after quiz	NOVEMBER 09, 2018	PASSED	
Check button functionality	NOVEMBER 09, 2018	PASSED	
Make sure prompts appear	NOVEMBER 09, 2018	PASSED	
Run through True or False Quiz successfully	NOVEMBER 09, 2018	PASSED	
Run Through Multiple Choice Quiz successfully	NOVEMBER 09, 2018		FAIL, THERE IS AN ERROR THAT OCCURS RANDOMLY
Checked timer did not get stuck	NOVEMBER 09, 2018	PASSED	

5.4. Test Plan Report

Testing for my application was done at every phase of the project. The reason that everything needed to be tested as I went along is because it is meant to be interactive and it requires that it does what it is supposed to do in order to run through the app. However, the testing that I did was once everything I believed to be right was complete. I went through and ran my app 5 times and checked specific functions of my application. From this I developed my table with the results. All of my test passed except one on the multiple-choice question. 2 out of the 5 times that I ran the app it kicked me out of the entire app and returned to XCode. I tried to debug and fix the problem but up to this point I am not sure what is causing the issue. It is not happening at a specific question or at a specific time.

My classmate gave me back great feedback. He had the same experience that I did with the testing. He did it 5 time and his results were also the same. However, it only messed up the simulation 1 out of the 5 times so that still proves an inconsistency. The only other thing that my classmate suggested is to think about making the app in Spanish or another language. In the future I would not mind implementing that and adding different quizzes to be utilized.

6. DETAILS

6.1. Challenges Overcome

Throughout the project I had issues that I had to resolve in order for my project to work correctly. One of the biggest challenges that I came across was towards the end of my project implementation. There was a major MacOS update in September that updated

my Macbook Pro from MacOS 10.13 High Sierra to MacOS 10.14 Mojave. This update also included an update to XCode which automatically updated it to the latest version available. This was problematic for me because the code that I was using worked on 9.4.1 but was changed and no longer available with XCode 10.0. I attempted to go back to the previous version of XCode, but it gave me a lot of issues because when I would open the application it would prompt me to update it. If I chose not to update it, I was unable to open up my files and work on my project since the update was mandatory. I resolved this issue by logging on to the Apple website and finding the download to the previous version that would not replace the one that was already active on my laptop. This felt like my biggest challenged because it caused me a lot of anxiety since it occurred after my project was already complete and ready to go.

Another challenge overcome was related to the questions that I needed for my Quiz. I have two options available for the user to test themselves on, Multiple Choice or True and False. The multiple-choice questions were easier to input into the plist file because the USCIS website has the questions that are going to be asked on the site with the wording that a tester would actually use while testing an applicant. Other websites also had the same questions in a multiple-choice format which made creating the options easier. I was unable to find the questions in true or false format, so I had to create my own. This was a bit challenging because a lot of them had to have several questions for just one of the multiple-choice questions.

6.2. Future Enhancements

In the future I would like to divide my application to have the same two test options but also have the option to choose how many questions you want to get tested on. Currently you have to run through the entire quiz in order to win or get a result. In the future I would like to focus on a smaller amount of questions so that way it is not as time consuming for the user. Another future enhancement that I would like to accomplish is changing it to have it record the ones that you get wrong and having that same question appear more than once while the Quiz is being taken. If this is implemented, then the user will have a better chance at remembering the question because they wouldn't have to wait until the entire quiz is complete to be able to see a question they missed again.

7. FINAL DEFENSE POWERPOINT

Civics Quiz App

October 09, 2018
Advisor: Valerie Sessions

Project Idea

- I was a derived citizen
- Becoming and American citizen opened a lot of opportunities for me
- Country of immigrants
- The United States benefits from people becoming U.S citizens

Citizenship Requirements

- Be at least 18 years old
- Be a permanent resident for at least 5 years
- Demonstrate continuous residence in the U.S for at least 5 years
- Show that you have lived for at least 3 months in the state or USCIS district where you apply
- Be able to read, write, and speak basic English
- Have a basic understanding of U.S history and government
- Be a person of good character
- Demonstrate an attachment to the principles and ideals of the U.S constitution

Xcode and Swift

- IDE for MacOS
- First released in 2003
- Used version 9.4.1
- Programming language developed by Apple Inc. for iOS, MacOS, tvOS, and Linux

View Controllers

- Add UIViewController as a subclass
  - Updating the contents of the views
  - Responding to user interaction with views
  - Coordinating objects
  - Resizing views and managing layout
- Multiple View Controllers

View Controller: Menu

- Added a Logo
- Gives both Quiz options
  - Multiple Choice
  - True or False
- Will display the High-score and Most Recent Score for quiz
- Consisted of mostly constraints

Menu Rules for Layout

```
let reglas = [

    //makes sure that it's using the entire content view.
    parteContenido.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor, constant: 0.0),
    parteContenido.leadingAnchor.constraint(equalTo: view.leadingAnchor),
    parteContenido.trailingAnchor.constraint(equalTo: view.trailingAnchor),
    parteContenido.bottomAnchor.constraint(equalTo: view.bottomAnchor),

    //
    parteLogo.topAnchor.constraint(equalTo: parteContenido.topAnchor, constant: 20.0),
    parteLogo.widthAnchor.constraint(equalTo: parteContenido.widthAnchor, multiplier: 0.6),
    parteLogo.centerXAnchor.constraint(equalTo: parteContenido.centerXAnchor),
    parteLogo.heightAnchor.constraint(equalTo: parteContenido.heightAnchor, multiplier: 0.2),

    parteBoton.topAnchor.constraint(equalTo: parteLogo.bottomAnchor, constant: 20.0),
    parteBoton.bottomAnchor.constraint(equalTo: partePuntacion.topAnchor, constant: -20.0),
    parteBoton.widthAnchor.constraint(equalTo: parteContenido.widthAnchor, multiplier: 0.6),
    parteBoton.centerXAnchor.constraint(equalTo: parteContenido.centerXAnchor),

    botonesDeQuiz[0].topAnchor.constraint(equalTo: parteBoton.topAnchor, constant: 0.0),
    botonesDeQuiz[0].bottomAnchor.constraint(equalTo: botonesDeQuiz[1].topAnchor, constant: -8.0),
    botonesDeQuiz[0].leadingAnchor.constraint(equalTo: parteBoton.leadingAnchor),
    botonesDeQuiz[0].trailingAnchor.constraint(equalTo: parteBoton.trailingAnchor),
    botonesDeQuiz[1].bottomAnchor.constraint(equalTo: parteBoton.bottomAnchor, constant: -8.0),
    botonesDeQuiz[1].leadingAnchor.constraint(equalTo: parteBoton.leadingAnchor),
    botonesDeQuiz[1].trailingAnchor.constraint(equalTo: parteBoton.trailingAnchor),
    botonesDeQuiz[0].heightAnchor.constraint(equalTo: botonesDeQuiz[1].heightAnchor),
```

Multiple Choice and T/F View Controllers

- View Controllers:
  - Sets layout
  - Load questions
  - Starts Timer
  - Checks and allows continuation to next question

```
@objc func botonRespuestaPresionado(_ sender: BotonRedondo) {
    reloj.invalidate()
    if sender.titleLabel?.text == preguntaCorriente.respuestaCorrecta {
        puntacion += 1
        etiquetaPregunta.text = "Click to continue"
        botonPregunta.isEnabled = true
    } else {
        sender.backgroundColor = rojoThanksgiving
        mal = mal + 1

        if(mal == 1){
            etiquetaPregunta.backgroundColor = colorDeLayerUno
        }
        else if(mal == 2){
            etiquetaPregunta.backgroundColor = colorDeLayerDos
        }
        else if(mal == 3){
            etiquetaPregunta.backgroundColor = colorDeLayerTres
        }
        else if(mal == 4){
            etiquetaPregunta.backgroundColor = colorDeLayerCuatro
        }
        else if(mal == 5){
            etiquetaPregunta.backgroundColor = colorDeLayerCinco
        }


        if(mal > 5){
            muestraAlerta(corRaro: 1)
        }
        etiquetaPregunta.text = "Click to continue"
        botonPregunta.isEnabled = true
    }
    for boton in botonesRespuestas {
        boton.isEnabled = false
        if boton.titleLabel?.text == preguntaCorriente.respuestaCorrecta {
            boto~.backgroundColor = verdeThanksgiving
        }
    }
}
```

Counter and Themes

- The quiz lets you get 5 questions incorrectly
- You have 30 seconds to answer a question
- The quiz will recored your highest score and your most recent score
- Correct and incorrect answers will display green and red
- Thanksgiving colored theme (Multiple Choice)
- Halloween colored theme (True or False)

Cocoa Touch Class

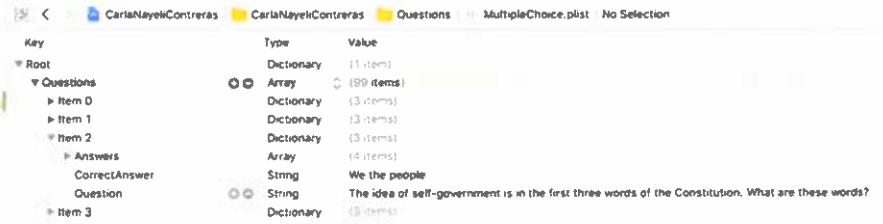
- UI framework for building software programs to run on iOS
- Serves as an abstraction layer of iOS
- Useful for subclassing classes from UIKit

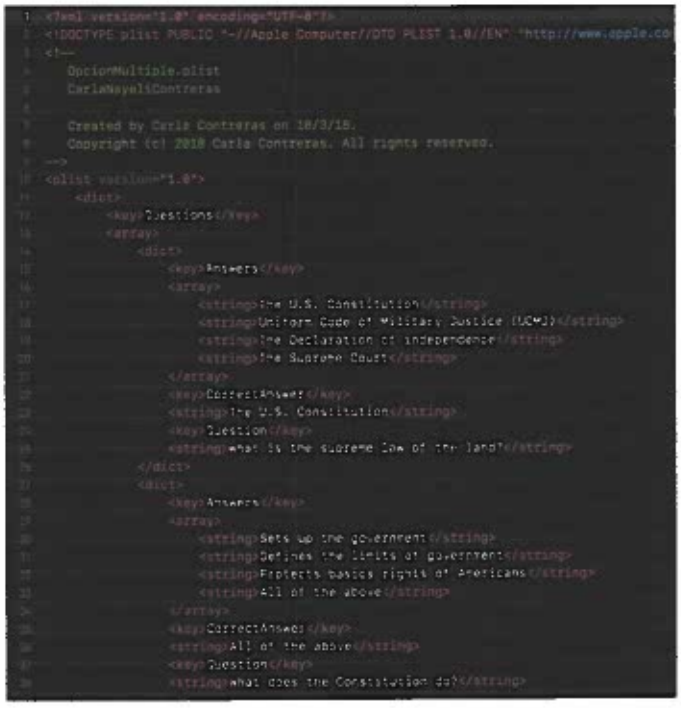
Swift - import Foundation

- Foundation framework provides base layer of functionally for apps including:
  - Data storage, text processing, sorting and filtering
- Example: Loader for quiz

Plist files

- XML file stored as a dictionary
  - Uses tags
- Apple's internal property list is a dictionary with key:value pairings but you can also use bool and array





Challenges Overcome

- I finished majority of my project before newest Xcode update came out last month
- Did not work with Xcode after 9.4.1
  - Had to revert back
- There are no true or false questions for the test online

Results